

РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

## Как завершить программный проект

Начать программный проект — это круто, но хорошо его завершить — признак зрелости вашего подхода к разработке.

7 августа 2023 года, 10:12 [Дэвид Истман](#)



Изображение через Unsplash

Независимо от того, идет ли речь о закрытии внутреннего проекта или публичной платформы, существует несколько способов завершить работу над программным обеспечением. Хотя **запуск программного проекта** в первый раз — это, безусловно, захватывающее занятие, грамотное завершение работы демонстрирует дополнительную зрелость вашего подхода к разработке. В этом посте я расскажу о том, на что стоит обратить внимание, если у вас есть немного времени и пространства для маневра. Возможно, это входит в ваши прямые обязанности, но, скорее всего, просто входит в сферу ваших интересов.

---

похоже, снизилось. Некоторые посетители ведут себя немного эксцентрично. Изменилась ценовая политика. Владельцу, похоже, все равно, но он пообещал, что заведение полностью преобразится. Оно сменило название на однобуквенное и повесило над дверью кричащую вывеску. Оглядываясь назад, вы понимаете, что что-то действительно закончилось, но никто в этом не признается.

Я, конечно, говорю не о ресторане, а о конкретной социальной сети. Глупо закрывать проект из-за того, что вы хотите сохранить пользователей, или просто потому, что не знаете, что сказать. Сейчас много говорят о том, что связь с клиентами теряется из-за того, что вы портите собственные продукты или услуги. **Энди Пайпер**, давний специалист по связям с разработчиками в Twitter, высказал интересную мысль о том, что закрытие должно было произойти раньше.

Большая ошибка — думать, что желание закончить отношения до того, как они закончатся, делает вас неудачником. Люди запоминают падения, но гораздо дольше они помнят несдержанное поведение, которое приводит к неудачам. И это касается в том числе тех, кто мог бы работать с вами.

## Увидь конец, прежде чем конец увидит тебя

Даже на самых комфортных концертах старайтесь придерживаться кодекса самурая (или бусидо); всегда будьте готовы умереть.

---

**Истинный смысл в том, что, постоянно осознавая свою смертность, люди могут достичь состояния свободы, выходящего за рамки жизни и смерти.**

---

One difference between a keen startup and a staid organization is that the former never sees or contemplates an end. And yet, this is not how the universe works. How do you combat this personally? In practice, this means as a developer being aware of decisions and actions that only make sense *If Nothing Bad Happens*. Don't deny risks out of fear, but imagine how the chips will fall if things do go South. That agreement that lasts longer than your project has even existed — is that sensible? How extended are we becoming? How long will it take to undo an action? What are the assumptions made about expected user numbers? Why are we depending on

---

unnecessary pessimism, it is just accepting the full lifecycle of a project may have a shorter timeline than was first imagined.

## A Good Ending

Some projects have ended with very good grace. [Thisismyjam.com](#) has a lovely memorial site, almost as good in death as it was in life. While this type of act of preservation is more devotion than good business practice, it would look good on a CV. The other extreme is Google, which has so many dead projects it has its own [software graveyard](#). But at least this sets expectations; you fail fast and move on. Your aim should just be to show pragmatism with a little empathy when an end comes.

### TRENDING STORIES

1. [Code is a message to the future](#)
2. [Code Anywhere: Turn Your Android Tablet Into a Dev Machine](#)
3. [Python for Beginners: When and How to Use Tuples](#)
4. [JetBrains is selling independence as the rest of AI coding picks sides](#)
5. [The Anatomy of Slow Code Reviews](#)

## The Why Doesn't Matter too Much

The actual reason for closing can simply be due to success, expense, or some other resource restriction. Or it can be that the company behind the product has changed direction. Or the facility will be covered elsewhere. But the true reason for closing may not be revealed to staff or customers anyway. In fact, it may remain shrouded for years. This is not something to worry about — even in your own life you have probably been a bit economical with the truth when explaining why you stopped doing something. Organizations have share value and competitors to think about, so a reasonable fiction is usually better than prolonged uncertainty. Either way, your responsibilities as a developer remain largely the same.

## Going, Going, Gone

---

are being decided, but no one is committed to anything. (I'm aware some organizations work like that permanently.) So whenever you plan, you have to caveat each point with whether you mean the left side of "end" before things cannot be reversed, or the right side of "end" when you have pointed your skies down the cliff and jumped.

Communicating intentions to users during the **sunsetting** period when an end is official is fraught because you can't help but treat them as a pack of scared gazelle who will bolt at the first sign of trouble. Yet most users have been stung by disappearing services before, so where possible just stick to saying what you know. Try to offer a way that users can escape and retrieve what they need. You do have to express things forcefully, otherwise, users will default to hoping they can continue as before. Ironically, if people are used to a good service, they will find it harder to understand what is going on.

## What About the Data

Sometimes you can just "give back" the data in terms of customer reports, but usually, something will be lost. A social network can give back a list of your tweets, but we all know it is the network that is of true value. In almost all cases held data should be safely disposed of, or it will become unmanaged. During the sunsetting period, try to offer the users as many escape options as you can reasonably support. This may be just an export of a comma-separated data file, or onboarding to another service entirely. A nice comprehensive blog post can at least notify users that you have thought about them. Note that services that don't lock users in have a natural advantage here.

## A List of Final Actions

The first time a project ends unexpectedly you won't be ready, but next time maybe you will. So this or any other list of points is not for rolling out upon requirement; consider these at all times and recognize when they occur elsewhere.

1. **Work out within your project team** whether employees will directly transfer to another project, or whether the closing of a project implies that the team will be disbanded. There may also be a small transition team to handle the changes. The trick here is to reduce the amount of uncertainty.
2. **Update stakeholders, and agree on a continuation plan.** This may be as little as supplying a set of fixed responses to certain queries or making sure any suggested transition will be supported. At the minimum, this is an email to a small set of people about the decisions and actions below.

carefully and make small alterations depending on how users respond to the news of the demise. But this whole area is a well-trodden topic by itself.

4. **Transfer, archive or destroy.** The project's software assets need to be dealt with in a systematic way. This just starts with turning everything off and responding to the consequences. Some assets should be handed off, and some deleted. Do not "open source" your code unless you already have a community who are ready and willing to maintain it. Otherwise, it is no more useful than fly tipping in GitHub, and will not ultimately help any stranded users.
5. **Complete administrative tasks.** There will be AWS relationships to curtail, domains or other external agreements to close off, etc. There will be some short unpleasant emails to send. And you know how Wikipedia updates things? That could be you doing that if your project is that well exposed.
6. **Hold that project post-mortem meeting.** Even if it is held informally years later, lessons learned should be formally recorded. If you can, try to do it earlier so that the enclosing organization has some memory of events, and doesn't just go down the same path. In this day and age, you may live through the end many times in a variety of podcasts and YouTube videos.

## And onto the Next Thing

This is the part where I say that after an end, there is a new beginning. I don't know how many interviewers take note of how their candidates ended projects — but they should. Even if you are impacted personally, try to do the right thing with your craft. That is, after all, what endures.

TNS



David has been a London-based professional software developer with Oracle Corp. and British Telecom, and a consultant helping teams work in a more agile fashion. He wrote a book on UI design and has been writing technical articles ever since....

[Read more from David Eastman →](#)

---

### SHARE THIS STORY



3. Python for Beginners: When and How to Use Tuples
4. JetBrains is selling independence as the rest of AI coding picks sides
5. The Anatomy of Slow Code Reviews

#### INSIGHTS FROM OUR SPONSORS



### Database Branching for AI Agents: How TINE Solves the Schema Drift Problem

15 June 2026

### Conway's Law in Reverse: Why AI Agents Need One Database, Not Ten

11 June 2026

### Build Persistent, Scalable AI Agent Memory with TiDB

10 June 2026



### Open Governance for MySQL: A Step Forward for the Community

25 June 2026

### Governing AI Assets at Scale with MCP Gateway and Registry

17 June 2026

### Introducing Trusted Remote Execution: Policy-Enforced Scripts for AI Agents and Humans

4 May 2026



---

## CEO Ash Kulkarni's organizational announcement to Elastic employees

24 June 2026

## How migrating to Elastic Security helped a digital safety software company cut incidents by 85% with UnderDefense

24 June 2026

## Elastic's no-code and full-code approaches to custom integrations

18 June 2026

---

### TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

---

## ARCHITECTURE

Cloud Native Ecosystem  
Containers  
Databases  
Edge Computing  
Infrastructure as Code  
Linux  
Microservices  
Open Source  
Networking  
Storage

## OPERATIONS

AI Operations  
CI/CD  
Cloud Services  
DevOps  
Kubernetes  
Observability  
Operations  
Platform Engineering

## THE NEW STACK

About / Contact  
Sponsors  
Advertise With Us  
Contributions

## FOLLOW TNS



## ENGINEERING

AI  
AI Engineering  
API Management  
Backend development  
Data  
Frontend Development  
Large Language Models  
Security  
Software Development  
WebAssembly

## CHANNELS

Podcasts  
Ebooks  
Events  
Webinars  
Newsletter  
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap  
Backend Developer Roadmap  
Devops Roadmap



---

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)